이제 실습입니다

파트 1 · 5교시 (14:00-14:50) · 중첩 자료구조 · 실습 ④
화면을 n8n으로 바꾸고, 프롬프터를 보며 진행합니다.

03

SKT K-뉴딜 아카데미 · AI·
자동화 기초 · 9/28 오후

중첩 자료구조
JSON

겹쳐 들어간 값을 읽고,
결과를 다른 프로그램이 읽을 수 있는 형식으로 남깁니다.

제각각인 줄이 한 모양으로 나옵니다

대소문자가 섞이고 칸이 모자란 로그를 한 모양으로 맞춰 파일로 남깁니다.

> 위가 들어가는 파일, 아래가 나오는 파일입니다.

```
# sample_logs_raw.csv (들어가는 것)
# 09:01,kim01,login_ok,10.0.3.21      ← 소문자
# 09:41,guest                          ← IP 가 없다
# normalized_logs.json (나오는 것)
```

```
[
  {"time": "09:01", "user": "kim01",
   "event": "LOGIN_OK", "ip": "10.0.3.21"},
  {"time": "09:41", "user": "guest",
   "event": "UNKNOWN", "ip": "unknown"}
]
```

오후에 나오는 낱말 네 개

용어 (한글발음)	한 줄 뜻	이 교시에서 보는 모양
중첩 nested, 네스티드	값 자리에 다시 딕셔너리나 리스트가 들어간 것	`{"user": {"id": "kim01"}}`
JSON 제이슨	프로그램끼리 데이터를 주고받기로 약속한 형식	`{"time": "09:12", "level": "WARN"}`
직렬화 serialize, 시리얼라이즈	메모리 속 값을 글자로 바꾸는 일	`json.dumps(값)`
필수 필드 required field	없으면 안 되는 항목	`time` · `user` · `event` · `ip`

JSON — 프로그램끼리 주고받기로 약속한 글자

값을 글자로 적어 둔 것입니다. 그래서 파일로 저장할 수도, 다른 프로그램에 보낼 수도 있습니다.

> **셋 다 모양이 같습니다.**
> **쓰이는 곳만 다를 뿐이에요 — 오늘 6교시에 첫째를, 9/30 에 셋째를 직접 받아 봅니다.**

```
# 로그 한 건
{"time": "23:50", "user": "gate01", "event": "LOGIN_FAIL"}


# 설정 파일
{"night_start": 0, "night_end": 6, "alert": true}


# 서버가 보내 주는 응답
```

이름으로 꺼내면 { }, 번호로 꺼내면 [ ]

기호	언제 쓰나	꺼낼 때
{ } 딕셔너리	값마다 이름이 필요할 때 {"user": "gate01"}	log["user"]
[ ] 리스트	같은 종류가 여러 개일 때 ["야간", "실패"]	tags[0]

> 값이 여러 개라고 리스트가 아닙니다 — 종류가 다르면 딕셔너리입니다.

```
{"time": "09:01", "user": "kim01", "event": "LOGIN_OK", "ip":
"10.0.3.21"}
```

실제 데이터는 한 겹이 아닙니다

왼쪽은 겹이 없는 모양, 오른쪽은 값 자리에 또 딕셔너리·리스트가 들어간 모양입니다.

지난주에 본 모양 — 납작하다

```
{"time": "23:50",
 "user": "gate01",
 "event": "LOGIN_FAIL"}
```

실제로 오는 모양 — 겹쳐 있다

```
{"time": "23:50",
 "user": {"name": "gate01",
          "dept": "영업"},
 "tags": ["야간", "실패"]}
```

겹친 값은 대괄호를 이어 붙여 꺼냅니다

앞 장의 오른쪽 모양을 `event` 라는 이름에 담아 두고 꺼내 봅니다.

> 왼쪽부터 한 겹씩 들어갑니다. 딕셔너리는 이름으로, 리스트는 번호로.

```
event = {"time": "23:50",
         "user": {"name": "gate01", "dept": "영업"},
         "tags": ["야간", "실패"]}

print(event["user"]["dept"])
```

```
영업
```

```
print(event["tags"][0])
```

중간 키가 하나라도 없으면 멈춥니다

없는 키를 꺼내면 그 자리에서 멈춥니다. 그래서 깊이 들어갈수록 있는지 먼저 확인합니다.

```
# 앞 장의 event 그대로
event = {"time": "23:50",
         "user": {"name": "gate01", "dept": "영업"},
         "tags": ["야간", "실패"]}

print(event["user"]["team"])
```

```
KeyError: 'team'
```

깊이 들어갈수록 있는지 먼저 봅니다

> 오전에 배운 `try` 로 감싸도 됩니다. 둘 다 맞고, 7교시 필수 필드 확인이 이 `in` 을 씁니다.

```
# 앞 장의 event 그대로
event = {"time": "23:50",
         "user": {"name": "gate01", "dept": "영업"},
         "tags": ["야간", "실패"]}
```

바로 꺼내면

```
print(event["user"]["team"])
```

```
KeyError: 'team'
```

`in` 으로 먼저 확인하면

```
if "team" in event["user"]:
    print(event["user"]["team"])
else:
    print("부서만 있습니다")
```

```
부서만 있습니다
```

우리가 쓸 모양은 리스트 안의 딕셔너리

로그 여러 건을 담으면 이 모양이 됩니다. 한 바퀴에 한 건을 꺼냅니다.

> 반복문을 돌면 `row` 에 딕셔너리 한 개가 들어옵니다. 그 안에서 이름으로 꺼냅니다.

```
logs = [
    {"time": "23:50", "user": "gate01", "event": "LOGIN_FAIL"},
    {"time": "02:40", "user": "hacker", "event": "LOGIN_FAIL"},
]
for row in logs:
    print(row["user"], row["event"])
```

```
gate01 LOGIN_FAIL
hacker LOGIN_FAIL
```

계정별 보고서는 딕셔너리 안의 딕셔너리

계정 이름이 키라서 `report["gate01"]` 한 번으로 꺼냅니다.
노트북은 `for user in report:` 로도 풉니다 — 결과는 같습니다.

```
report = {
    "gate01": {"fail": 3, "ip": "10.0.5.9"},
    "hacker": {"fail": 1, "ip": "10.0.9.9"},
}
for user, info in report.items():
    print(user, info["fail"])
```

```
gate01 3
hacker 1
```

9/30 API · 10/6 LLM 응답도 이 모양입니다

언제	무엇이 겹쳐 오나
오늘 6교시	JSON 파일 — 리스트 안에 딕셔너리
9/30 API	조회 응답 — `data["connection"]["isp"]`
10/6 LLM	모델 응답 — 세 겹까지 들어간다

> 문법은 오늘 배운 대괄호 이어 붙이기 하나뿐입니다. 데이터만 달라집니다.

실습 ④ · 노트북 5교시 절 · 02_0928_pm_nested_json.ipynb

문제 5-1 ~ 5-13 · 도전 ⭐5-7 · ⭐5-14

① **사본을 저장하고 `logs` 셀을 실행합니다**

5교시 내내 그 `logs` 를 씁니다

② **문제 5-1 ~ 5-6 — 리스트 안의 딕셔너리**

계정 이름을 모두 출력하고, 실패 건수를 세면 성공

③ **문제 5-8 ~ 5-13 — 딕셔너리 안의 딕셔너리**

계정마다 실패 횟수를 출력하고, 한 건을 추가하면 성공

같은 한 건인데 적는 법이 다릅니다

왼쪽은 파이썬만 아는 모양이라 다른 언어가 못 읽습니다.
그래서 오른쪽 글자로 바꿔 건넵니다.

파이썬이 화면에 보여 주는 모양 — 작은따옴표 · True · None

```
{'user': 'gate01',
 'night': True,
 'ip': None}
```

파일에 적히는 JSON — 큰따옴표 · true · null

```
{"user": "gate01",
 "night": true,
 "ip": null}
```

파이썬은 따옴표 둘 다 됩니다 — 작은따옴표는 파이썬이 따옴표까지 보여 줄 때의 모양입니다. JSON 은 큰따옴표만 허용합니다.

값 안에 들어 있을 때만 따옴표가 보입니다

```
print("가나다")
```

```
가나다
```

```
print({"a": "값"})
```

```
{'a': '값'}
```

```
print({"note": "안에 '작은따옴표'가 있으면"})
```

```
{'note': "안에 '작은따옴표'가 있으면"}
```

> 코드에 적을 때는 둘 다 자유 · 화면에 따옴표까지 보일 때는 작은따옴표가 기본 · JSON 파일은 큰따옴표만.

이 파일을 3·4과목이 그대로 읽습니다

우리 파서
파이썬

→

normalized_logs.json
글자로 바꾼 것

→

접근통제 · 이상탐지
3·4과목

주고받는 형식만 맞추면 안쪽을 무엇으로 만들든 이어 붙습니다 — 11월 캡스톤이 이 구조입니다.

내용은 같습니다 — 딕셔너리는 값, JSON 은 글자

헷갈리면 `type()` 으로 확인합니다. 답은 둘 중 하나입니다.

> 중괄호가 보인다고 다 JSON 이 아닙니다. `json.dumps` 를 지난 것만 JSON입니다.
> 딕셔너리는 `alert["user"]` 로 꺼내지지만, JSON 글자는 안 꺼내집니다 — 먼저 `loads` 로 되돌려야 합니다.

```
alert = {"user": "gate01", "count": 2}    # 딕셔너리
text = json.dumps(alert)                          # JSON

print(type(alert))
```

```
<class 'dict'>

print(type(text))
```

JSON과 딕셔너리는 비슷하지만 4가지가 다릅니다

견주는 점	JSON	파이썬 딕셔너리
참과 거짓	`true` / `false`	`True` / `False`
값이 없음	`null`	`None`
따옴표	큰따옴표만 된다	작은따옴표도 된다
정체	파일이나 글자 한 덩이	메모리 안의 값

> 읽어 들이는 순간 파이썬 표기로 바뀝니다. `true` 는 `True` 가 됩니다.

JSON 이 쓰이는 자리 셋

자리	무엇	언제 만나나
API 응답	서버가 보내 주는 답	9/29 · 9/30
설정 파일	프로그램이 읽는 값들	10/2 스케줄러
과목끼리 전달	오늘 만드는 것	10월 3·4과목

> 한 가지 형식을 배워 두면 세 자리에서 다 쓰입니다. 그래서 오늘 시간을 들입니다.

이름이 비슷한 명령 네 개

끝에 `s` 가 붙으면 글자를 다루고, 안 붙으면 파일을 다룹니다.

명령	하는 일	언제 쓰나
`json.dump`**`s`**`(값)`	값을 JSON 글자로 편다	API 로 보낼 때
`json.load`**`s`**`(글자)`	JSON 글자를 값으로 되돌린다	API 응답을 받을 때
`json.dump(값, f)`	값을 파일에 저장한다	결과를 남길 때
`json.load(f)`	파일을 읽어 값으로 되돌린다	남긴 것을 다시 쓸 때

여기서 s 는 string 입니다 — 파이썬 설명문도 `dumps` 는 str 로 돌려준다, `dump` 는 파일에 쓴다로 적혀 있습니다.

dumps — 값을 글자로 바꿉니다

.dumps 뜻: 값을 JSON 글자로 편다

한글이 \uc57c 같은 기호로 바뀌었습니다.
기본 설정이 아스키만 쓰기 때문에 한글이 깨졌습니다 — 다음 장에서 고칩니다.

```
import json
alert = {"user": "gate01", "note": "야간 실패", "count": 2}
print(json.dumps(alert))
```

```
{"user": "gate01", "note": "\uc57c\uac04 \uc2e4\ud328", "count": 2}
```

한글을 저장하면 이상한 글자로 나옵니다

고장이 아닙니다. `json` 이 기본으로 영문·숫자·기호만 쓰도록 되어 있어서 그렇습니다.

> 둘 다 프로그램이 읽는 데는 문제가 없습니다.
> 다만 `\uc57c` 는 사람이 못 읽으니, 우리가 열어 볼 파일에는 항상 붙입니다.

```
alert = {"note": "야간 실패"}

print(json.dumps(alert))
```

```
{"note": "\uc57c\uac04 \uc2e4\ud328"}
```

```
print(json.dumps(alert, ensure_ascii=False))
```

```
{"note": "야간 실패"}
```

적는 것	뜻	한글이

저장할 때 옵션 두 개를 항상 붙입니다

> `ensure_ascii=False` 는 한글을 한글로, `indent=2` 는 줄을 맞춰 사람이 읽게 합니다.

```
alert = {"user": "gate01", "note": "야간 실패", "count": 2}
```

옵션 없이

```
json.dumps(alert)
```

```
{"user": "gate01",
 "note": "\uc57c\uac04 …"}
```

옵션 두 개를 붙이면

```
json.dumps(alert,
    ensure_ascii=False, indent=2
)
```

```
{
  "user": "gate01",
  "note": "야간 실패"
}
```

dump — 파일로 남깁니다

```python
import json
alert = {"user": "gate01", "note": "야간 실패", "count": 2}    # 아직 딕셔너리

# "w" 는 쓰기 — 파일이 없으면 만들고, 있으면 덮어쓴다
with open("alert.json", "w", encoding="utf-8") as f:
    # dumps 와 달리 s 가 없다 — 글자를 돌려주지 않고 f 에 바로 쓴다
    json.dump(alert, f, ensure_ascii=False, indent=2)
```

(화면에는 아무것도 안 나옵니다)

```
# alert.json 을 열어 보면
```

```
{
  "user": "gate01",
```

loads — 받은 글자를 값으로 되돌립니다

> 파일 안에서는 `true` 였는데 읽어 들이니 `True` 입니다. 표기는 파일 안에서의 이야기입니다.

```
import json

# 서버가 보내 준 글자라고 하자 — 따옴표로 감싼 한 덩이다
text = '{"user": "gate01", "fail": 3, "night": true}'
back = json.loads(text)
print(back["fail"])
```

```
3
```

```
print(back["night"])
```

```
True
```

여러 건을 한 파일에 담을 때도 `json.dump` 하나면 됩니다

앞에서는 딕셔너리 한 건을 담았습니다. 이번에는 리스트 통째로 담습니다.

> 두 건이라고 `dump` 를 두 번 부르지 않습니다. 리스트 하나를 한 번에 넘깁니다.
> 파일 첫 글자가 대괄호면 여러 건, 중괄호면 한 건입니다.

```
rows = [{"user": "gate01"}, {"user": "hacker"}]

# rows 를 rows.json 이라는 파일에 담는다
with open("rows.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, ensure_ascii=False, indent=2)

# rows.json 을 열어 보면
```

```
[
  {"user": "gate01"},
  {"user": "hacker"}
]
```

load — 파일을 읽어 값으로 되돌립니다

> 읽어 들이면 다시 딕셔너리입니다. 그래서 이름으로 꺼낼 수 있습니다

```
with open("alert.json", encoding="utf-8") as f:
    back = json.load(f)
print(back["user"])
```

```
gate01
```

작은따옴표로 적은 JSON 은 `json.loads` 가 읽지 못합니다

> 메시지가 「큰따옴표로 감싼 이름을 기대했다」고 말해 줍니다. JSON 은 큰따옴표만 됩니다

```
json.loads("{'user': 'admin'}")
```

```
JSONDecodeError: Expecting property name
enclosed in double quotes: line 1 column 2
```

저장한 파일은 `!cat` 으로 열어서 확인합니다

화면에 아무것도 안 나오는 작업은 열어 보지 않으면 잘 됐는지 알 수 없습니다.
저장 → 열기를 한 쌍으로 붙입니다.

```
# 코랩에서 파일 내용 보기 — 터미널 명령
!cat rows.json
```

```
[
  {"user": "gate01"},
  {"user": "hacker"}
]
```

실습 ⑤ · 노트북 6교시 절 · 02_0928_pm_nested_json.ipynb

문제 6-1 ~ 6-13 · 도전 ⭐6-7 · ⭐6-14

① **문제 6-1 ~ 6-6 — 글자 쪽**

한글이 기호로 바뀌는 것을 보고, 옵션으로 고치면 성공

② **문제 6-8 ~ 6-11 — 파일 쪽**

파일로 저장하고 다시 읽어 값이 나오면 성공

③ **문제 6-12 · 5-13 — 로그 목록을 파일로**

여러 건을 한 파일에 담고, 읽어서 건수를 세면 성공

같은 사건인데 모양이 다릅니다

시스템, 장비마다 대소문자가 다르고, 어떤 줄은 칸이 모자랍니다.

```
# sample_logs_raw.csv
09:01,kim01,login_ok,10.0.3.21        ← 소문자
09:05,admin,LOGIN_FAIL,10.0.9.8       ← 대문자
09:41,guest                            ← 칸이 두 개
```

> 이 상태로 「LOGIN_FAIL 인 건수」를 세면 소문자 줄이 빠집니다.
> 세기 전에 모양을 맞춰야 합니다.

맞추지 않고 세면 6건이 1건이 됩니다

> 에러가 나지 않습니다. 조용히 1건으로 보고합니다 — 오전에 배운 그 「조용한 실패」입니다.

```
# sample_logs_raw.csv 11줄에서 실패를 센다
# 모양을 맞추지 않고 "LOGIN_FAIL" 만 센 결과
```

1건

```
# 전부 대문자로 맞춘 뒤 센 결과
```

6건

사건 이름은 대문자로 맞춥니다

> 어느 쪽으로 맞출지는 사람이 정합니다. 우리는 대문자로 정했고, 정했으면 전부 그 규칙을 따릅니다.

```
event = "login_fail"
print(event.upper())
```

```
LOGIN_FAIL
```

```
# 이미 대문자인 값은 그대로 있습니다
print("LOGIN_OK".upper())
```

```
LOGIN_OK
```

없는 칸은 unknown 으로 채웁니다

> 버리지 않고 칸을 채워 남깁니다. 「IP 를 모른다」도 알아야 하는 정보입니다.

그냥 꺼내면

```
parts = "09:41,guest".split(",")
print(parts[3])
```

```
IndexError
```

길이를 먼저 보면

```
parts = "09:41,guest".split(",")
if len(parts) >= 4:
    ip = parts[3]
else:
    ip = "unknown"
```

```
unknown
```

줄 끝에 눈에 안 보이는 것이 붙어 옵니다

파일을 한 줄씩 읽으면 줄 끝에 줄바꿈 문자가 붙습니다. `strip()` 이 그것을 떼어 냅니다.

```
line = "09:41,guest\n"           # 파일에서 읽은 줄
print(line.split(",")[1] == "guest")
```

```
False
```

```
print(line.strip().split(",")[1] == "guest")
```

```
True
```

한 줄을 한 모양으로 만듭니다

강조한 두 줄이 앞 장의 `if·else` 입니다. IP 를 먼저 정해 두고 딕셔너리에 넣습니다.

```
line = "09:41,guest\n"          # 파일에서 읽은 줄
parts = line.strip().split(",")
if len(parts) >= 4:
    ip = parts[3]
else:
    ip = "unknown"
row = {"time": parts[0], "user": parts[1],
       "event": parts[2].upper(), "ip": ip}
```

```
{'time': '09:41', 'user': 'guest', 'event': 'UNKNOWN', 'ip': 'unknown'}
```

약속을 지켰는지 확인합니다

없으면 안 되는 항목을 목록으로 적어 두고, 하나씩 있는지 봅니다.

> **`not in`으로 확인하고, 무엇이 빠졌는지를 모아 둡니다.**
> 「걸렀다」만 남기면 왜 걸렀는지 모릅니다.

```
# 앞 장에서 만든 row
row = {"time": "09:41", "user": "guest", "event": "UNKNOWN"}

required = ["time", "user", "event", "ip"]
missing = []
for key in required:
    if key not in row:
        missing.append(key)
print("빠진 항목:", missing)
```

normalize_logs.py — 완성된 모양

> 저장이 반복문 밖에 한 번만 있습니다. 안에 두면 매 줄마다 덮어써서 마지막 한 건만 남습니다.

```
import json
rows = []
for line in open("sample_logs_raw.csv", encoding="utf-8"):
    parts = line.strip().split(",")
    if len(parts) >= 2:
        rows.append({"time": parts[0], "user": parts[1], …})

with open("normalized_logs.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, ensure_ascii=False, indent=2)
```

저장을 반복문 안에 두면 한 건만 남습니다

> "w" 는 덮어쓰기라서 매 바퀴 파일을 갈아 버립니다.

```
rows = [{"user": "kim01"}, {"user": "admin"}]
f = open("out.json", "w", encoding="utf-8")

for row in rows:
    json.dump(row, f)
```

```
{"user": "admin"}
— 마지막 한 건만 남는다
```

반복문 밖에서 한 번 저장하면 전부 남습니다

목록에 모아 두고 마지막에 한 번 저장합니다.

```
rows = [{"user": "kim01"}, {"user": "admin"}]
f = open("out.json", "w", encoding="utf-8")
out = []

for row in rows:
    out.append(row)
json.dump(out, f)
```

```
[{"user": "kim01"}, {"user": "admin"}]
— 전부 남는다
```

나온 파일은 다음 과목이 읽습니다

대소문자가 맞고 칸이 다 있습니다.
이 모양만 약속해 두면 3과목·4과목이 안쪽을 어떻게 만들든 이어 붙습니다.

```
# normalized_logs.json
-----------------------------------------------------------------------------
[
  {"time": "09:01", "user": "kim01",
   "event": "LOGIN_OK", "ip": "10.0.3.21"},
  {"time": "09:41", "user": "guest",
   "event": "UNKNOWN", "ip": "unknown"}
]
```

몇 번 돌려도 같은 결과여야 합니다

무엇	두 번 돌리면	왜 그런가
`normalized_logs.json`	같다	`"w"` 로 갈아 쓴다
`agent.log` (오전)	늘어난다	로깅은 덧붙인다

> 둘의 성질이 다릅니다. 결과 파일은 갈아 쓰고, 로그는 쌓입니다. 헷갈리면 파일이 이상해집니다.

실습 ⑥ · 노트북 7교시 절 · 02_0928_pm_nested_json.ipynb

문제 7-1 ~ 7-13 · 도전 ⭐7-8 · ⭐7-14

① **맨 위 데이터 셀을 실행합니다**

`sample_logs_raw.csv` 가 만들어집니다 (오전 파일과 다른 이름)

② **문제 7-1 ~ 7-7 — 한 모양으로 맞추기**

소문자를 대문자로, 없는 IP 를 `unknown` 으로. 6-1·6-2 는 먼저 답을 적는 문제

③ **문제 7-9 ~ 7-13 — `normalize_logs.py` 와 드라이브**

`agent_core` 에 `.py` 와 `.json` 이 보이면 성공. 6-9·6-10 는 먼저 답을 적는 문제

오늘 만든 것 네 개를 확인합니다

드라이브 agent_core 폴더를 열어 네 개가 다 있는지 봅니다.

파일	무엇	언제 만들었나
log_parser.py	깨진 줄이 있어도 멈추지 않는 파서	오전 4교시
agent.log	파서가 남긴 로그	오전 4교시
normalize_logs.py	로그를 한 모양으로 맞추는 프로그램	오후 7교시
normalized_logs.json	다음 과목이 읽을 결과 파일	오후 7교시

사흘 만에 파일 네 개가 생겼습니다

날	만든 것	무엇을 할 수 있게 됐나
9/22	`day01_basic.py`	값을 담고 꺼낸다
9/23	함수 · CSV 읽기	파일을 읽어 센다
오늘 오전	`log_parser.py` · `agent.log`	깨져도 멈추지 않고 로그를 남긴다
오늘 오후	`normalize_logs.py` · `normalized_logs.json`	다음 과목에 넘긴다

남은 ⭐도전 문제를 모아 푸는 시간

넘어간 문제를 지금 풉니다. 다 풀지 않아도 됩니다.

어디	도전 문제
5교시	⭐5-7 실패한 계정 목록 · ⭐5-14 로그에서 보고서 만들기
6교시	⭐6-7 목록 전체를 JSON 글자로 · ⭐6-14 파일에서 실패만 세기
7교시	⭐7-8 야간 실패만 고르기 · ⭐7-14 계정별 보고서도 남기기

> 오전 도전 문제(⭐2-1~2-5 · ⭐3-1~3-5 · ⭐4-1~4-5)가 남은 분은 그것부터 해도 됩니다.
> 하나라도 더 해 보는 것이 목적입니다.

오늘 막혔던 자리 셋

① **저장했는데 화면에 안 나온다**

정상입니다. `!cat` 으로 파일을 열어 확인합니다

② **파일에 한 건만 있다**

저장이 반복문 안에 있습니다. 모아서 마지막에 한 번

③ **`KeyError` 가 난다**

겹을 하나 더 들어가려 했습니다. 중간까지 `print` 해 봅니다

오늘 배운 것 한눈에

오전 — 죽지 않고 로그를 남긴다

- `try·except` — 멈추지 않는다
- 예외 이름으로 나눠 잡기 · `finally`
- `logging` — 화면이 아니라 파일로

오후 — 주고받을 수 있게 남긴다

- 리스트 안의 딕셔너리 · 딕셔너리 안의 딕셔너리
- JSON 네 명령 — `dumps·loads·dump·load`
- 정규화 · 필수 필드 확인

내일 9/29은 정규표현식입니다. 쉼표조차 없는 원본 로그를 같은 모양으로 만듭니다.

이번 주에 남은 것

날	무엇	오늘 것을 어떻게 쓰나
9/29 (화)	정규표현식 · 탐지 룰 셋	쉼표 없는 로그를 같은 모양으로 만든다
9/30 (수) 오전	API 로 실제 조회	탐지한 IP 가 어디 것인지 물어본다
10/2 (금)	웹훅 · 스케줄러	여기까지를 부르지 않아도 돌게 만든다

나흘 뒤에는 사람이 누르지 않아도 도는 프로그램이 됩니다.
오늘 만든 파일이 그 안에 그대로 들어갑니다.

끝나기 전에 — 숙제 하나

오늘 깔 것은 없습니다. 계정 하나만 만들어 오면 됩니다.

① **숙제 — 깃허브 계정을 만듭니다**

모레 9/30 아침에 씁니다.
아이디는 이력서에 적을 주소가 됩니다

② **내일 아침 — 폴더와 경로**
· 깃배시로 `cd`·`mkdir` 를 칩니다

③ **10/2(금) 1교시 — VS Code**
· 내 노트북에 편집기를 깝니다.
그날 같이 합니다

🔔 **17:30 — 여기서 수업을 마치고 오후 리추얼로 넘어갑니다.**
리추얼 20분(17:30~17:50) 뒤 17:50 에 끝납니다. 늦게 끝내지 않습니다.